# Lab 11 — Why Kubernetes: from Compose to the cluster

*Theory — what Compose will never be able to do, what a cluster is, why you describe a state there instead of issuing commands, and how k3s puts all of that into your WSL in one minute.*

## Objectives

- Name the limits of Compose that justify an orchestrator.
- Describe a cluster: *control plane* and nodes, and the role of each component.
- Understand the **declarative model** and the **reconciliation loop**.
- Recognise the Podman pod from lab 07 in the Kubernetes Pod.
- Install k3s on WSL, talk to it with `kubectl`, and get a Podman image into the cluster.

---

## 1. What Compose cannot do

In lab 09, `podman compose up -d` brought up the whole stack on your machine. In lab 10, you saw what was missing for production: if the machine goes down, everything goes down; if the API needs three instances on three machines, Compose has no idea; if one version must replace another without cutting the service, you script it yourself; and if a container dies, `restart` relaunches it — in the same place, with the same failure. Those four gaps have names: **high availability**, **distribution**, **progressive update**, **self-healing**. An orchestrator exists to provide them.

> **→ HISTORY**
> Google has run its services in containers since 2003 with an internal system, **Borg**, which schedules millions of tasks over hundreds of thousands of machines. In 2014, three Google engineers (Joe Beda, Brendan Burns, Craig McLuckie) published an open rewrite of it: **Kubernetes** (Greek for *helmsman*, abbreviated "k8s"). Donated in 2015 to the Cloud Native Computing Foundation, it became the de facto standard among orchestrators within five years, ahead of Swarm and Mesos. **k3s** (Rancher, 2019) is a certified, lightweight distribution for the edge, IoT and development workstations.

## 2. A cluster: one brain and many arms

A **cluster** is a set of machines (*nodes*) that run containers, driven by a **control plane** that decides where and how.

| Component | Role | The Podman analogy |
|---|---|---|
| **API server** | the only front door: receives, validates and stores every request | the `podman` CLI, but as an HTTPS server |
| **etcd** | key-value store holding the desired and the observed state | Podman's local database |
| **scheduler** | picks the node for each Pod | nobody: your own machine |
| **controller manager** | the loops that compare desired with observed, and correct | `--restart`, generalised to everything |
| **kubelet** (on every node) | the agent that runs the requested Pods | `podman` + `conmon` |
| **container runtime** (containerd, through CRI) | actually creates the containers | `crun` |
| **kube-proxy** (on every node) | the network of the Services (lab 13) | `rootlessport` |

Every exchange goes through the **API server**: `kubectl`, dashboards, controllers, the kubelet itself. That is what makes Kubernetes extensible — anything that speaks HTTPS can drive it — and why the first question of any diagnosis is "does the API server answer?" (`kubectl cluster-info`).

> **K3S**
> k3s packs **the whole control plane into a single 70 MB binary**, replaces etcd with SQLite (enough for one node), embeds containerd, and adds what a bare cluster lacks: an Ingress controller (Traefik), a load balancer (ServiceLB), a storage class (`local-path`), CoreDNS and metrics-server. It runs as **root**, as a systemd service — the kubelet creates the containers, not you. An enterprise cluster spreads those components over several machines; the commands and the manifests are **the same**.

## 3. The declarative model

With Podman, you give **orders**: `run`, `stop`, `rm`. With Kubernetes, you submit a **description**: "I want a Pod named `api`, with this image, this port, this variable". The API server stores it; the controllers and the kubelet work until reality matches; and they **keep** watching. If the Pod disappears, the gap between desired and observed is detected and corrected. That is the **reconciliation loop**, the heart of all of Kubernetes.

An object is described in YAML with four keys: `apiVersion` (the API group and version), `kind` (the type: Pod, Deployment, Service…), `metadata` (name, namespace, labels), `spec` (what you want). The cluster adds `status` (what it observes). `kubectl apply -f file.yaml` sends the spec; `kubectl get pod api -o yaml` returns spec **and** status.

> **REMEMBER**
> `kubectl apply` is **idempotent**: run ten times, it changes nothing if the file has not changed, and it updates if the file has changed. `kubectl create` creates once and fails the second time (`AlreadyExists`). A versioned manifest + `apply` = the same discipline as Compose, applied by a system that never stops checking.

In lab 11, the lone Pod is still an object **without a controller**: deleted, it does not come back. It is the Deployment (lab 12) that carries the loop "I want N replicas". You will see it at the end of the hands-on lab: a Pod erased under a Deployment reappears within seconds.

## 4. The Pod

The unit of deployment is not the container but the **Pod**: one or more containers sharing an IP address, a `localhost`, and volumes. You already made one in lab 07 with `podman pod create`: same concept, same word, same shared `/etc/hosts`. A Pod receives an IP from the cluster network (`10.42.0.x` with k3s); it changes with every recreation, and no client must hard-code it — that will be the job of Services (lab 13).

> **→ DOCKER RECAP**
> A Podman container = an isolated process; a Podman pod = several containers in the same network namespace; `podman generate kube` writes that pod as Kubernetes YAML and `podman kube play` reads it back. The Kubernetes Pod is exactly that object, placed on a node by a scheduler and watched by a kubelet. What you knew about containers stays true inside: PID 1, signals, exit codes, logs on stdout, environment variables.

> **→ NETWORK**
> Each node receives an address range for its Pods (`10.42.0.0/24` on the first k3s node), and a **CNI** (*Container Network Interface*, here flannel) links the nodes' ranges together: every Pod reaches every Pod by IP, with no NAT. It is the user-defined network from lab 07 extended to several machines. The host keeps its own address (`hostIP` in the Pod's status).

In practice, most Pods have **only one** container; the second is a *sidecar* (log agent, TLS proxy) or an *init container* (schema migration before the API).

## 5. `kubectl` and the kubeconfig

`kubectl` reads a **kubeconfig** file (`~/.kube/config`, or `KUBECONFIG`, or for k3s `/etc/rancher/k3s/k3s.yaml`) holding the API server's address, the authority's certificate and your credentials; a **context** ties together a cluster, a user and a default **namespace**. Namespaces partition objects; in this series, every lab works in its own (`kubectl create namespace lab11`, then `kubectl config set-context --current --namespace=lab11`).

| Command | Use |
|---|---|
| `kubectl get pods` (`-o wide`, `-o yaml`, `-A`) | list, with node and IP, or the full object, or all namespaces |
| `kubectl describe pod api` | details + **Events**: the timeline of what the cluster did |
| `kubectl logs api`, `kubectl exec api -- cmd` | like `podman logs` / `podman exec` |
| `kubectl apply -f`, `kubectl delete -f` | submit / withdraw manifests |
| `kubectl run web --image=…` | create a Pod with no file (quick test) |
| `kubectl port-forward pod/api 18080:8080` | local tunnel to a Pod, with no published port |
| `kubectl explain pod.spec.containers` | the documentation of every field, offline |
| `kubectl api-resources` | every object type of the cluster |

→ WINDOWS / WSL

One-line install, inside your Ubuntu: `curl -sfL https://get.k3s.io | sudo sh -s - --write-kubeconfig-mode 644`. The script installs the binary, a `k3s` systemd service (so `systemd=true` is mandatory) and a `kubectl`. `--write-kubeconfig-mode 644` makes the kubeconfig readable without `sudo`: convenient on a workstation, unacceptable on a shared server (that file **is** administrator access to the cluster). Budget 4 GB of RAM for WSL (`.wslconfig`, `memory=6GB` recommended); `wsl --shutdown` stops the cluster, systemd relaunches it on the next `wsl`.

## 6. Getting an image into the cluster

The cluster has **its own** image store, that of k3s's containerd (as root, in `/var/lib/rancher/k3s`). Podman has its own (`~/.local/share/containers`). A `podman build` therefore gives the cluster nothing: `kubectl apply` of a Pod on `localhost/api:1.0` ends in `ErrImagePull` then `ImagePullBackOff`, because the kubelet tries to **pull** the image from a registry called `localhost`. Two solutions: import it (`podman save localhost/api:1.0 | sudo k3s ctr images import -`), which this series does; or push it to a registry the cluster knows (lab 17). Either way, `imagePullPolicy: IfNotPresent` tells the kubelet to use the local image if it exists; `Always` (the default for the `latest` tag) would force a pull at every start.

PODMAN

Podman speaks Kubernetes natively: `podman generate kube` writes your pods as YAML, `podman kube play` reads a Kubernetes manifest and creates the matching pods with no cluster. The generated file does contain machine-specific things (`hostPath`, `hostPort`) that a cluster replaces with ConfigMaps and Services. And `sudo k3s crictl ps` / `sudo k3s crictl images` show the containers and images **of the cluster**, where `podman ps` only shows yours.

## 7. In the workplace

- **Developers write manifests, the platform team runs the cluster.** You will receive a kubeconfig with one context per environment and one namespace per team; you will never see the control plane.
- **A managed cluster** (EKS, AKS, GKE) or OpenShift replaces k3s in production: control plane run by the provider, nodes on demand, load balancers and storage wired to the cloud. Your manifests do not change.
- **Compose stays on the workstation**; Kubernetes begins at the first shared environment. Many teams keep both, and a `kubectl apply` in CI does what `compose up` does on the workstation (lab 17, GitOps).
- **Vocabulary matters**: Pod, Deployment, Service, Ingress, namespace, manifest, reconciliation. It is the vocabulary of incidents, reviews and interviews.

---

## Remember

- Compose does neither high availability, nor distribution, nor progressive updates, nor self-healing: that is an orchestrator's job.
- A cluster = a control plane (API server, etcd, scheduler, controllers) + nodes (kubelet, containerd, kube-proxy); everything goes through the API server.
- Kubernetes is **declarative**: you submit a desired state, reconciliation loops maintain it; `apply` is idempotent.

- The Pod is the unit of deployment — the Podman pod from lab 07 — with an ephemeral IP from the cluster network.
- `kubectl` reads a kubeconfig, works in a context and a namespace; `describe` and its Events are the first diagnostic tool.
- The cluster has its own image store: `podman save | sudo k3s ctr images import -`, and `imagePullPolicy: IfNotPresent`.
- k3s = all of Kubernetes in one binary, with Traefik, ServiceLB and `local-path`; the manifests are those of any cluster.

## Vocabulary

**orchestrator**: system that places, watches and repairs containers across several machines. — **cluster / node**: set of machines / one machine that runs Pods. — **control plane**: API server, etcd, scheduler, controller manager. — **kubelet**: node agent that runs the Pods. — **CRI / containerd**: the node's container interface and runtime. — **manifest**: YAML file describing an object (`apiVersion`, `kind`, `metadata`, `spec`). — **desired / observed state**: `spec` / `status`. — **reconciliation loop**: controller that corrects the gap between the two. — **Pod**: group of containers sharing IP, `localhost` and volumes. — **CNI**: network plugin that links Pods across nodes. — **kubeconfig / context / namespace**: cluster access / cluster + user + default namespace / partition of objects. — **`imagePullPolicy`**: when the kubelet pulls an image. — **k3s**: certified Kubernetes distribution in one binary.